

PROYECTO SQL-BASE DE DATOS SAKILA

1. Introducción

Este informe documenta el trabajo realizado sobre la base de datos **Sakila**. Esta base de datos modela el negocio de una cadena de tiendas de alquiler de películas: catálogo de películas, actores, categorías, inventario, alquileres, pagos, clientes, empleados y tiendas. Todo esto puede verse en las tablas que la forman y las relaciones existentes entre ellas.

El objetivo del proyecto es demostrar el manejo completo de SQL utilizando **DBeaver** como herramienta de gestión y consulta: desde operaciones básicas sobre una única tabla hasta consultas que combinan varias tablas mediante relaciones, subconsultas, vistas reutilizables y estructuras de datos temporales.

El informe no se limita a mostrar el código SQL de cada consulta. Por cada una se explica **qué pregunta de negocio responde, por qué se ha elegido esa técnica** (JOIN frente a subconsulta, vista frente a consulta directa, etc.) y **qué dice el resultado obtenido**.

2. Análisis previo de la base de datos

- 2.1 ESQUEMAS Y RELACIONES

Para ver el diagrama Entidad-Relación ver el documento *.png* que se encuentra en el repositorio.

La base de datos Sakila se organiza en tres bloques de tablas relacionadas entre sí mediante claves foráneas:

1. **Catálogo de películas:** **film** es la tabla central de este bloque. Se relaciona con **actor** a través de la tabla **film_actor** (relación N:M, ya que una película tiene varios actores y un actor participa en varias películas), y con **category** a través de **film_category**. Además, **film** tiene una relación 1:N con **language**, ya que cada película tiene un único idioma pero un idioma puede corresponder a muchas películas.

2. **Gestión de alquiler:** `inventory` representa las copias de cada película en cada tienda. Cada copia puede alquilarse varias veces a lo largo del tiempo, lo que se registra en `rental` (relación 1:N desde `inventory`). Cada alquiler genera a su vez un registro en `payment` (relación 1:N desde `rental`, y también desde `customer` y `staff`, ya que un pago está asociado a un cliente concreto y a un empleado que lo gestiona).
3. **Personas y ubicaciones:** `customer` y `staff` comparten una estructura similar: ambas se relacionan 1:N con `address`, que a su vez se relaciona 1:N con `city`, y esta con `country`. Cada `store` tiene asignado un `manager_staff_id` que la vincula con un empleado responsable. Esta estructura combina relaciones **1:N** (la mayoría de los casos, como `rental` → `customer`) con relaciones **N:M** resueltas mediante tablas puente (`film_actor`, `film_category`).

- 2.2 VOLUMEN DE DATOS

Para poder ver el número de registros que hay en las tablas principales. A partir de ahora se copia las consultas directamente de DBeaver y el resultado obtenida:

TABLA	REGISTROS
payment	16049
rental	16044
film	1000
customer	599
actor	200
category	16

3. Consultas sobre la base de datos

- 3.1 CONSULTAS BÁSICAS (1 TABLA)

1. Películas más largas de 120 minutos, ordenadas por duración

Para poder ver el número de registros que hay en las tablas principales. A partir de ahora se copia las consultas directamente de DBeaver y el resultado obtenida:

```
select title, length, rental_rate
from film
where length > 120
order by length desc;
```

	A-Z title	length	rental_rate
1	HOME PITY	185	4.99
2	GANGS PRIDE	185	2.99
3	DARN FORRESTER	185	4.99
4	WORST BANGER	185	2.99
5	SWEET BROTHERHOOD	185	2.99
6	CONTROL ANTHEM	185	4.99
7	POND SEATTLE	185	2.99

2. Número de películas por clasificación

```
select rating, count(*) as num_peliculas
from film
group by rating
order by num_peliculas desc;
```

	A-Z rating	num_peliculas
1	PG-13	223
2	NC-17	210
3	R	195
4	PG	194
5	G	178

3. Top 5 películas más caras de reponer

```
select title, replacement_cost
from film
order by replacement_cost desc
limit 5;
```

	A-Z title	123 replacement_cost
1	BLINDNESS GUN	29.99
2	BONNIE HOLOCAUST	29.99
3	ARABIA DOGMA	29.99
4	BALLROOM MOCKINGBIR	29.99
5	CHARIOTS CONSPIRACY	29.99

- 3.2 CONSULTAS RELACIONES ENTRE TABLAS

1. Consultar información de los clientes: Hacer unión entre las tablas que contienen las información de los diferentes clientes de la tienda para tener un registro con sus nombres completos, ciudad y dirección detalladas.

```
select c.first_name, c.last_name, ci.city, co.country
from customer c
inner join address a on c.address_id = a.address_id
inner join city ci on a.city_id = ci.city_id
inner join country co on ci.country_id = co.country_id
order by co.country, ci.city;
```

	A-Z first_name	A-Z last_name	A-Z city	A-Z country
1	VERA	MCCOY	Kabul	Afghanistan
2	MARIO	CHEATHAM	Batna	Algeria
3	JUDY	GRAY	Bchar	Algeria
4	JUNE	CARROLL	Skikda	Algeria
5	ANTHONY	SCHWAB	Tafuna	American Samoa
6	CLAUDE	HERZOG	Benguela	Angola
7	MARTIN	BALES	Namibe	Angola

2. Top 10 clientes que más han gastado

```
select c.customer_id, c.first_name, c.last_name, sum(p.amount) as gasto_total
from customer c
inner join payment p on c.customer_id = p.customer_id
group by c.customer_id, c.first_name, c.last_name
order by gasto_total desc
limit 10;
```

	123 customer_id	AZ first_name	AZ last_name	123 gasto_total
1	526	KARL	SEAL	221.55
2	148	ELEANOR	HUNT	216.54
3	144	CLARA	SHAW	195.58
4	178	MARION	SNYDER	194.61
5	137	RHONDA	KENNEDY	194.61
6	459	TOMMY	COLLAZO	186.62

3. Registro de clientes con deudas con sus datos: Interesante ya que hay algunos clientes que no han realizado la devolución de sus préstamos

```
select r.rental_id, r.rental_date, c.first_name, c.last_name, c.email
from rental r
inner join customer c on r.customer_id = c.customer_id
where r.return_date is null
order by r.rental_date;
```

	123 rental_id	rental_date	AZ first_name	AZ last_name	AZ email
1	14,098	2005-08-21 00:30:32.000	DWAYNE	OLVERA	DWAYNE.OLVERA@sakilacustomer.org
2	11,541	2006-02-14 15:16:03.000	GREGORY	MAULDIN	GREGORY.MAULDIN@sakilacustomer.org
3	11,563	2006-02-14 15:16:03.000	LOUISE	JENKINS	LOUISE.JENKINS@sakilacustomer.org
4	11,577	2006-02-14 15:16:03.000	WILLIE	HOWELL	WILLIE.HOWELL@sakilacustomer.org
5	11,593	2006-02-14 15:16:03.000	EMILY	DIAZ	EMILY.DIAZ@sakilacustomer.org
6	11,611	2006-02-14 15:16:03.000	LAURIE	LAWRENCE	LAURIE.LAWRENCE@sakilacustomer.org

4. Ingresos totales por tiendas

```
select s.store_id, sum(p.amount) as ingresos_totales
from payment p
inner join staff st on p.staff_id = st.staff_id
inner join store s on st.store_id = s.store_id
group by s.store_id
order by ingresos_totales desc;
```

123 store_id	123 ingresos_totales
2	33,927.04
1	33,489.47

5. Categorías con menos copias disponibles en inventario

```
select cat.name as categoria, count(i.inventory_id) as copias_disponibles
from category cat
left join film_category fc on cat.category_id = fc.category_id
left join inventory i on fc.film_id = i.film_id
group by cat.name
order by copias_disponibles asc;
```

	AZ categoria	123 copias_disponibles
1	Music	232
2	Travel	235
3	Horror	248
4	Comedy	269
5	Children	269
6	Classics	270

- 3.3 SUBCONSULTAS

1. Películas que nunca han sido alquiladas por clientes

```
select f.title
from film f
where not exists (
  select 1
  from inventory i
  join rental r on r.inventory_id = i.inventory_id
  where i.film_id = f.film_id);
```

AZ title
ALICE FANTASIA
APOLLO TEEN
ARGONAUTS TOWN
ARK RIDGEMONT
ARSENIC INDEPENDENCE
BOONDOCK BALLROOM

2. Clientes cuyo gasto total supera el gasto medio de todos los clientes

```
select c.customer_id, c.first_name, c.last_name, sum(p.amount) as gasto_total
from customer c
join payment p on c.customer_id = p.customer_id
group by c.customer_id, c.first_name, c.last_name
having sum(p.amount) > (
  select avg(gasto) from (
    select sum(amount) as gasto
    from payment
    group by customer_id) sub ) order by gasto_total desc;
```

123 customer_id	AZ first_name	AZ last_name	123 gasto_total
526	KARL	SEAL	221.55
148	ELEANOR	HUNT	216.54
144	CLARA	SHAW	195.58
178	MARION	SNYDER	194.61
137	RHONDA	KENNEDY	194.61
459	TOMMY	COLLAZO	186.62

3. Última película alquilada por cada cliente

```
select distinct on (c.customer_id)
  c.customer_id,
  c.first_name,
  c.last_name,
  r.rental_date as ultimo_alquiler,
  f.title as ultima_pelicula
from customer c
left join rental r on r.customer_id = c.customer_id
left join inventory i on i.inventory_id = r.inventory_id
left join film f on f.film_id = i.film_id
order by c.customer_id, r.rental_date desc;
```

	123 customer_id	AZ first_name	AZ last_name	ultimo_alquiler	AZ ultima_pelicula
1	1	MARY	SMITH	2005-08-22 20:03:46.000	BIKINI BORROWERS
2	2	PATRICIA	JOHNSON	2005-08-23 17:39:35.000	OPEN AFRICAN
3	3	LINDA	WILLIAMS	2005-08-23 07:10:14.000	LUCKY FLYING
4	4	BARBARA	JONES	2005-08-23 07:43:00.000	REUNION WITCHES
5	5	ELIZABETH	BROWN	2006-02-14 15:16:03.000	GABLES METROPOLIS
6	6	JENNIFER	DAVIS	2005-08-23 06:41:32.000	MASKED BUBBLE

- 3.4 VISTAS

Vista 1: Obtener el total por cliente, cuántos alquileres ha realizado el mismo y cuánto ha gastado cada uno.

```
create view vista_gasto_clientes as
select
  c.customer_id,
  c.first_name,
  c.last_name,
  count(distinct r.rental_id) as num_alquileres,
  sum(p.amount) as gasto_total
from customer c
left join rental r on r.customer_id = c.customer_id
left join payment p on p.customer_id = c.customer_id
group by c.customer_id, c.first_name, c.last_name;
```

Vista 2: Tener un inventario de el stock que hay de cada una de las películas en las tiendas.

```
create view vista_inventario_disponible as  
select  
    s.store_id,  
    f.film_id,  
    f.title,  
    count(i.inventory_id) as copias_totales  
from film f  
join inventory i on i.film_id = f.film_id  
join store s on s.store_id = i.store_id  
group by s.store_id, f.film_id, f.title;
```

- 3.5 TABLAS TEMPORALES

* verse en el documento *05_temporales.sql*